

AST326Y1 Practical 1 Report: Basic Photon Statistics

LUNA SARRAZIN,¹ ÖMER CAN CANER,¹ GUILLAUME LAMOTTE,¹ AND COOPER LEWIS¹

¹*AST326Y1 2026-2027 Group H, University of Toronto*

ABSTRACT

Complementary metal oxide semi-conductor (CMOS) cameras are becoming an increasingly popular option for astronomical observations. The Point Grey Firefly FMVU-03MT series of CMOS detectors (present at the University of Toronto) however does not provide measurements for conversion gain (G), read noise σ_R , or dark current σ_{DC} in its user manual. In this report, we begin by observing the properties of the monochrome Firefly sensors from the aforementioned series (FMVU-03MTM) and how exposure time (shutter speed), framerate, and aperture individually affect the observed image: lengthening exposure time shifts ADU values to the right on the viewer histogram (a "brighter" image) and increases the gap between different brightness peaks (a singular peak is formed at the lowest exposure times); an increasing framerate decreases the maximum exposure time, and increasing framerate with the lens cap on the camera decreases the mean ADU value on the histogram (a "darker" image); narrowing aperture shifts histogram bins to the left (decreasing ADU values), merges histogram peaks into a larger singular peak, and produces a significantly crisper and clear image. We then use images captured with the camera to find a gain of $87.2 \text{ e}^-/\text{ADUs}$, read noise of 74.4 e^- , and dark current of $2,753 \text{ e}^- \text{ s}^{-1} \text{ px}^{-1}$.

1. INTRODUCTION

1.1. *Motives*

One of the core parts of astronomy is to record observations. Various cameras have made their appearance over time to simplify this process and yield more precise and accurate results. Charge-coupled devices (CCDs) are still a common pick for observations, but complementary metal oxide semi-conductor (CMOS) cameras have been taking their place as a popular choice for these tasks.

CMOS cameras convert photoelectron detections into Analog-to-Digital Units (ADUs) using an Analog-to-Digital Converter (ADC) within each of the camera's pixels. There are

common errors caused by noise to consider with measurements: read noise, denoted σ_R , a normally-distributed type of noise originating from the camera hardware itself; and dark current, denoted σ_{DC} , a type of Poisson-distributed noise originating from the thermal excitation of electrons being read as additional brightness values. σ_{DC} . Dark current is a type of shot noise (σ_{SH} , originating from factors external to the camera. The conversion gain (G) of the camera is the number of photoelectrons that need to be detected to cause an increase in one analog-to-digital unit (ADU).

The University of Toronto provides a variety of cameras to choose from for AST326Y1: Practical Astronomy. We use a Firefly monochrome CMOS camera (FMVU-03MTM) during this lab, and focus on two objectives: understand-

ing how to use such cameras because of their ubiquity in astronomical observations, aiding us in developing valuable skills for later experiments and research; and measuring the conversion gain, σ_R , and σ_{DC} of our camera, as these values are not included in the camera’s manual. This will allow future users of this camera model to make faster conversions from ADUs to electrons, and have estimations of the random errors from noise in their measurements.

Our observations and data of the CMOS camera’s behaviour and values for later sections are presented in Section 2, the methods we employ for data reduction in Section 3, and our analysis and modelling of the data in Section 4. We discuss our results in Section 5.

2. OBSERVATIONS & DATA

2.1. *Aside: Statistics in Python*

First and foremost, we introduce ourselves to performing statistics calculations in Python. This segment is coded entirely by myself within Jupyter Hub for its ease of use.

We first begin by recreating *Figure 7* from the lab handout. This is done by hardcoding the list into Python directly. The mean and standard deviation are calculated using the NumPy library, and the Poisson distribution calculated with our own implementation with help from SciPy’s `factorial` method. All plots are made using `matplotlib.pyplot`.

Figure 1 compares the handout histogram (top) to our histogram (bottom). The data and its presentation are very similar, to the exception of the location of the ticks (in the centre of bins for the handout plot, but to the left of bins for our plot). This also affects the location of the Poisson pmf on our histogram, which is slightly shifted to the left compared to the handout figure. The main segment of the code used for this portion can be found in Appendix A.

Then, we expand our plotting to files containing photon readings from a star, both with short

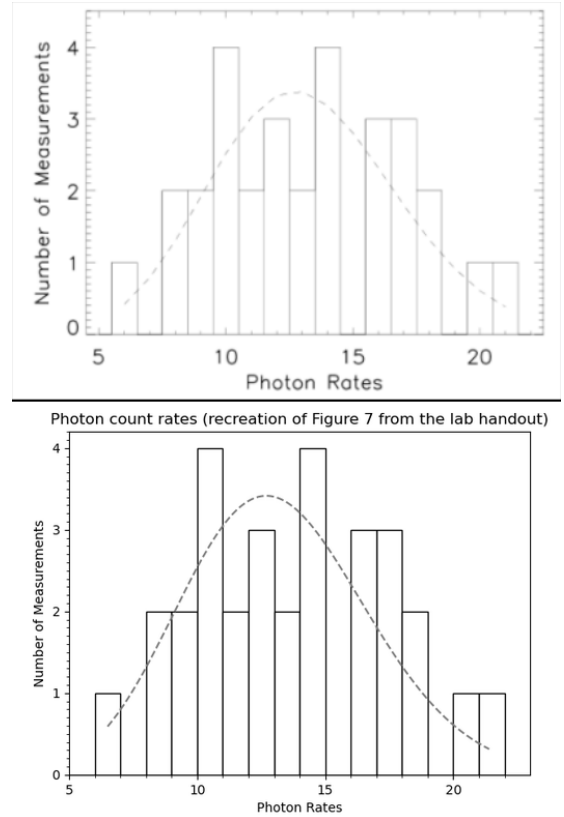


Figure 1. Lab 1 handout figure 7 (top), and recreation of the figure using NumPy and `matplotlib.pyplot` (bottom)

(`small.txt`) and long (`large.txt`) integration times. The data files can be found in the main branch of this group’s GitHub repository,² in the `P1/src/4-1` directory.

Figure 2 presents two histograms obtained with the data from `small.txt` (top) and `large.txt` (bottom), each overplotted by their mean (magenta vertical line) and a Poisson (blue) and Gaussian (orange) distributions using the data’s means and standard deviations. The Poisson and Gaussian distributions were implemented in-house; the code can be found in Appendix B.

We notice that `small.txt` is better approximated by a Poisson distribution given the skewed shape of the histogram. This is further

² <https://github.com/flypt7/AST326-Practicals>

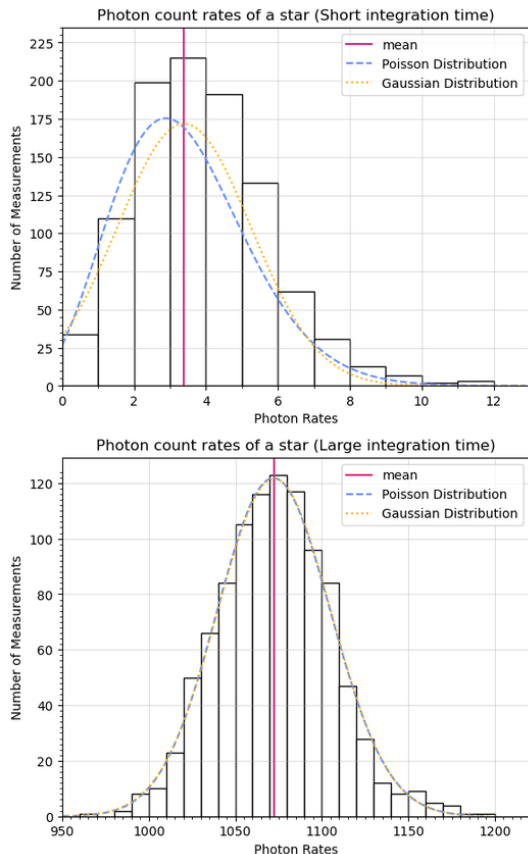


Figure 2. Lab 1 handout figure 7 (top), and recreation of the figure using NumPy and matplotlib.pyplot (bottom)

confirmed by the mean-variance relation of the Poisson distribution:

$$\mu \approx \sigma^2 \quad (1)$$

Where μ is the mean and σ^2 is the variance (or standard deviation squared). We find a mean of $\mu_{small} = 3.392$ ADUs and a standard deviation $\sigma_{small} = 1.858$ ADUs, so $\sigma_{small}^2 \approx \mu_{small}$ similar to the Poisson distribution.

As for `large.txt`, the data is approximated by both a Poisson and Normal distribution, as the two overlap closely. That is because of the fact that the Poisson distribution tends to a Normal distribution as $\mu \rightarrow \infty$.

2.2. Exposure Time and Saturation

While the Firefly MV03-MTM has a 10-bit ADC, our observations only take 8 bits into con-

sideration by design of the viewer program we used. Therefore, the maximum ADU value we can obtain is 255 ADUs ($2^8 - 1$, similar to any 8-bit ADC).

Measurements for this section were made by the entire group during the first lab session, on September 17, 2026, between 18:00 and 19:00 (UTC-4:00), in the Sidney Smith Hall SS1072 room.

For this segment, we begin by setting up the camera facing toward a black thermos water bottle with stickers alongside a laptop, with a small amount of wall visible in the background. We then launch the lab viewer, turn all auto settings off and set the view to full. The gain is set to 9.72 dB, brightness to 146, framerate to 30 fps, and gamma to 0.

With that, we initially set the exposure time (equivalent to shutter speed) to 0.033 seconds. The chosen scene produces very dark images, with a mean of 35.134 ADUs and values ranging between approximately 10 ADUs and 100 ADUs. We can distinguish multiple peaks on the histogram, from the varying colours of each object in the camera's view (lower peaks from the bottle and laptop, brighter ones from the white outline of the sticker and the wall in the background). We do not have any saturated pixels, nor pixels at 0 ADU.

When decreasing the shutter speed, the entire camera view becomes progressively darker, and histogram peaks start getting closer to each other, until effectively forming one very large peak at approximately 6.5 ADUs at the minimum shutter speed ($40\mu\text{s}$). Some very rare pixels still have higher values than the bulk, some up to approximately 20 ADUs. These rare pixels likely have two primary causes: they can be camera hot spots (pixels that erroneously detect more photoelectrons than there are); or some of the brighter objects such as the bottle sticker white edges and the wall are still being detected

with more photoelectrons. We look further into the former in the following section.

No pixels are saturated at this minimum exposure time, but we do notice 0.001% of pixels at 0 ADUs. These are likely not caused by dark columns, which would have been noted with higher shutter speeds as well. So, this fraction of the pixels is perhaps detecting too few photoelectrons to have a non-zero value.

Overall, the reason why the image is generally very dark is because the f-stop was set really narrow for our measurements. This causes a significant reduction in the amount of photoelectrons received by pixels. Section 2.4 delves into this subject in more detail.

2.3. *In the Dark!*

Measurements for this section were made by the entire group during the first lab session, on September 17, 2026, between 18:00 and 19:00 (UTC-4:00), in the Sidney Smith Hall SS1072 room.

We attempt to look for detection anomalies from the camera. This primarily includes hot spots (pixels that detect significantly more photoelectrons than there are), dark columns (of pixels not detecting any light, visible as black columns on images). We do so by putting the lens cap back onto the camera, setting the viewer to full display so that it shows the actual brightness of each pixel, and start with framerate at its lowest allowed value (6.90518 frames/s). The maximum shutter speed at this framerate is 0.27s. The quasi-totality of the pixels are dark, and the viewer program reports a mean brightness of 15.9 ADUs. There are also brighter pixels and areas which only comprise a very low minority of the camera output.

When increasing the framerate from lowest to highest, we notice that the maximum shutter speed decreases significantly. The brightness of each pixel also decreases, and brighter pixels become dimmer but remain noticeable. Once we reach the maximum framerate (45.632

frames/s), the maximum shutter speed is 0.02s, and the mean brightness has dropped to 10.5 ADUs, approximately a 34% decrease. In other words, individual bright pixels become brighter as exposure lengthens (analogous to decreasing framerate).

The brighter pixels present regardless of framerate are most likely camera hot spots. In other CMOS- and CCD-related experiments, these would be checked for in advance and ruled out of data processing and calculations.

2.4. *Aperture Modulations*

Measurements for this section were made by the entire group during the first lab session, on September 17, 2026, between 18:00 and 19:00 (UTC-4:00), in the Sidney Smith Hall SS1072 room.

In an attempt to uncover the effects of aperture, we decide to point the camera at the uniform wall in the SS1072 room. We set the aperture to F/1.4, the gain to 12dB (the maximum on our CMOS camera), and modulate the shutter until obtaining a mean of 193 ADUs. This yields a standard deviation of 9.5 ADUs.

When narrowing the aperture (going closer to F/8), we notice that the histogram of pixel ADU measurements starts shifting to the left (lower ADU values) and becomes thinner; pixels are getting dimmer but also more homogeneous. When reaching an aperture near F/C (closed aperture), we observe a mean of 11.2 ADUs and a standard deviation of 1.6 ADUs; the mean decreases to nearly 5% of its value with the widest aperture, and the standard deviation to approximately a sixth.

We repeat this process with a non-uniform target: a set of chairs and the classroom wall in the background.

Because of the variety in colour of the objects, we ensure that there was little to no saturation with aperture F/1.4, and manage to bring it down to 0.008%. We obtain two peaks on the histogram: one in the lower ADU end (approx-

imately 20 ADUs) from the dark brown chairs, and another in the higher range (170-210 ADU) from the bright grey walls. The wall and chairs were discernable, although details from the wall are challenging to discern. The mean brightness was 91.411 ADUs, and the standard deviation 84.373 ADUs. This very large standard deviation is expected given the fact that we only have two peaks at opposite ends of the histogram.

Narrowing the aperture has the same effect as with a uniform source: a general decrease in ADU values, with peaks becoming narrower and closer to each other. With an aperture of F/8, we obtain a mean of 65.426 ADUs and a standard deviation of 60.535 ADUs. We do however also obtain a clearer image, with the a clearer separation between the wall and the chairs, and crisp details on the wall. The reason for this is that the aperture also acts as a light collimator: the narrower the aperture, the more parallel the photoelectrons detected will be, resulting in a crisper image with clear details and little to no blur. However, this also means that less light is allowed through, leading to an overall darker image. The opposite effect occurs when the aperture is widened: less collimated photoelectrons are also detected, causing the image to become brighter but also blurrier.

2.5. Read Noise and Conversion Gain Frames

Measurements for this section were made by the whole group during the second lab session, on September 24, 2026, between 18:00 and 19:00 (UTC-4:00), in the Sidney Smith Hall SS1072 room. The capture sets were recorded at 18:49 for the light images, and 18:51 for the dark images. Note that the camera used for this section and all data processing linked to it was verified to be the same model CMOS as previous sections (FMVU-03MTM), but it is not the exact same camera.

In order to measure the read noise of the camera, we need to record two sets of 30 images: the first from a target with a variety of brightnesses;

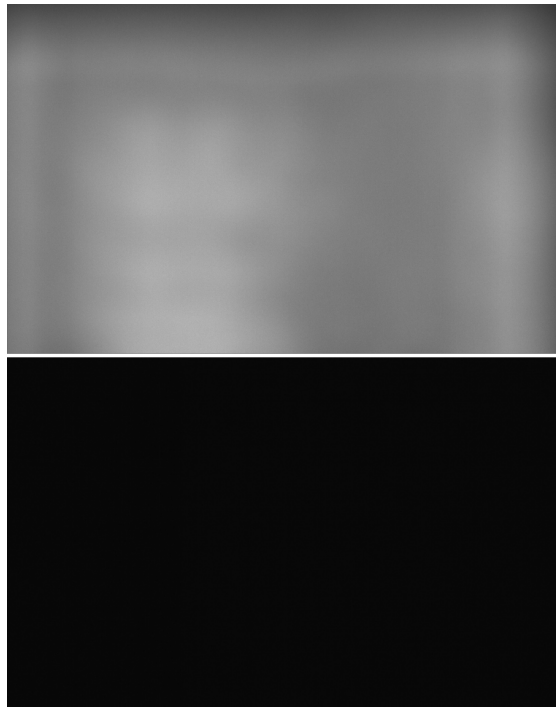


Figure 3. Viewer program image of the first image from the sets made with laptop sticker (top) and with lens cap on (bottom). Brightness 180, exposure 40, frame rate 45.6279fps, gain 0dB, gamma 0, shutter 0.00541598s for both.

and the second with the same parameters as the first but with the lens cap on the camera. We want to ensure that dark frames have a non-zero brightness, so we put the lens cap on the camera to view its ADU values, and modify the parameters accordingly. The parameters for this section are as follows: 180 brightness, 45.6fps framerate, 0dB gain, 0 gamma, and 5.4ms shutter speed. These settings are kept constant for capturing all the frames in this section.

We choose the target for the light set to be a sticker on the back of Guillaume’s laptop because of its variety in brightness peaks, which we smooth into a broader range of values by slightly blurring the image. This allows for more consistent data reduction in later sections. We record the 30 light frames, and place the lens cap back on the camera to record the 30 dark frames.

Figure 3 shows the first images of each set of 30 from the target (top) and with the lens cap on (bottom). The image sets are created with the capture executable as follows: `lab1-capture --dir light_groupH --n 30` for the light set, and `lab1-capture --dir dark_groupH --n 30` for the dark set. The main parameters are listed in the figure’s caption, and all parameters for both sets can be found in the `settings.txt` file created with each set, included in Appendix C.

Note that this section is performed with another CMOS camera than all previous sections, which is why the dark frame has no anomalous bright pixels in the presented dark frame.

2.6. Dark Current Frames

Measurements for this section were made by the whole group during the third lab session, on October 1, 2026, between 18:00 and 19:00 (UTC-4:00), in the Sidney Smith Hall SS1072 room. More precise timestamps are given for the camera captures. Note that the camera used for this section and all data processing linked to it was verified to be the same model CMOS as all previous sections (FMVU-03MTM), but it is not the exact same camera. I led the data capturing for this section, with help from group members in verifying the camera settings.

In order to measure the camera’s dark current, we must capture images from the camera with the lens cap on at varying shutter speeds. To do so, we use the lab viewer program provided to us, with brightness set to 146, gain to 12 dB, framerate to 30fps, and gamma to 0. We take 5 measurements at 7 increasing shutter speeds of 1ms, 5ms, 10ms, 15ms, 20ms, 25ms, and 30ms, so as to ensure we can rule out potential outliers while still having analyzable data. Captures are made similarly to the ones in Section 2.5, replacing the last argument from 30 to 5 (for 5 frame captures).

We expect to see an increasing relationship between the mean brightness and shutter speed, as

longer exposure time allows more light to be detected per interval. While the images themselves seem to not show much difference to the naked eye, the `settings.txt` file for each set of captures also lists the mean ADU, as well as minimum and maximum ADU values over all captures in that set. We notice that there is a general increasing pattern for the mean brightness between 1ms and 5ms shutter speeds (from 9.468 to 9.588 ADUs), but that this mean then decreases to values below 5 ADUs for 10ms, 15ms, 20ms, and 25ms, and shoots back up to 10.389 ADUs at 30ms exposure time. This is most likely caused by some outliers in the data from the middle shutter speeds, which is confirmed by their `settings.txt` files: the minimum values for each of them are near zero, meaning that some pixels might have had an effect similar to a shutdown on some frames. This is further developed upon in Section 3.2 when the data is filtered, and its potential causes are discussed in Section 5.2. An excerpt from the `settings.txt` file generated with the 1ms shutter speed samples is included in Appendix F.

3. DATA REDUCTION & METHODS

3.1. Read Noise and Conversion Gain Method

To find the read noise of our camera, we use the sets of frames mentioned in Section 2.5. There are multiple processing steps to acquire read noise and conversion gain, namely:

1. Calculate the average brightness of the all dark frames combined with `numpy.mean`.
2. Subtract this value from each pixel in each of the bright frames. We call the resulting frames ”diff frames”.
3. Calculate the mean and variance of each pixel in the diff frames using `NumPy` methods, and make a scatter plot of them on a mean-variance plot (mean is the x-axis, variance the y-axis)
4. Plot a linear regression of the scatter plot.

Thus, we begin by reading each image file using the Python Image Library (PIL) into NumPy 2D arrays, and stacking the arrays of both sets of 30 images into one 3D array each (the light array and the dark array). We then calculate the mean per pixel from the dark array, and stack it on itself to create a new 3D array of the same shape as the set of light frames to directly subtract it from the light frames, giving us our diff frames.

We calculate the mean and variance of the diff frames using their respective NumPy methods, and flatten the results into a 1D array for random pixel selection; for calculation speed and because of sufficiently large sample size, we decide to randomly select 2,500 samples from the original 360,960 pixels using a NumPy `Generator` class instance. A seed was set to the instance to ensure consistency of data selection throughout code runs.

The random points are then appended to a new NumPy 1D array, and we perform least squares regression using an in-house implementation. The algorithm solves the optimization matrix-vector multiplication computationally using all values in the array, and returns the slope and intercept of the best-fit linear equation. The full (but short!) algorithm is presented in Appendix D. We also calculate an R^2 value for the best-fit line to attest the fit’s accuracy, using an in-house method closely based on the correlation coefficient (R) calculation in the SciPy statistics library’s `linregress` function. The code for this computation can be found in Appendix E.

The framework code for data processing in this section was developed by myself and shared with other group members for their use and modification.

3.2. Dark Current Method

There are multiple steps to measure the dark current of the camera from our recorded frames captured in Section 2.6:

Table 1. Output of printing the 2D array of mean brightness per frame (3 d.p.) at increasing exposure times over five trials. Outliers in bold red print.

Shutter	T1	T2	T3	T4	T5
1ms	9.474	9.467	9.479	9.456	9.462
5ms	9.578	9.584	9.599	9.592	9.587
10ms	9.809	9.791	9.777	0.042	9.791
15ms	9.938	9.931	9.932	0.043	0.043
20ms	10.087	10.073	10.089	0.043	0.043
25ms	10.220	10.210	10.245	0.046	0.044
30ms	10.377	10.402	10.399	10.382	10.384

1. Calculate the mean ADU value for each frame at each shutter speed.
2. Aggregate the mean of each shutter speed, including uncertainties, using their 5 recorded frames.
3. Scatter the aggregate mean points against shutter speed in seconds.
4. Plot a linear regression line of the scatter plot.

Thus, we begin by reading all images using the Python Image Library (PIL), and immediately calculate their mean ADU values using `numpy.mean`. Mean values from each shutter speed are appended to their own 1D NumPy arrays, which are then combined into a single NumPy 2D array.

We do notice some potential outliers when viewing the `settings.txt` files for each shutter speed, namely at speeds of 10ms, 15ms, 20ms, and 25ms. We check for the presence of these potential outliers in our calculated means by printing the aforementioned 2D array, for which the output is shown in Table 1. There are indeed specific frames with near-zero mean ADU values, which is not expected given how these frames were recorded. We address these outliers and their potential sources in Section 5.2.

We decide to filter these values out by excluding brightness values below 5 ADUs from any further calculations; other frames show very little variation in mean ADU values, so we believe that any value that significantly diverges from other frames should be ignored. We then calculate an aggregate mean (a mean of the mean values, using `numpy.mean`). Read noise is a normally-distributed source of uncertainty, so we also estimate an uncertainty U for each shutter speed using the following formula:

$$U = \sigma_R / \sqrt{N} \quad (2)$$

Where σ_R is the read noise of the camera, and N is the number of samples. This calculation is performed for every aggregate mean we calculate. Despite not using the exact same camera between the read noise and dark current measurements, we assume that the two cameras behave similarly and have comparable G and σ_R .

Both the aggregate means and uncertainties are appended to new NumPy 1D arrays. We then scatter the aggregate means over their shutter speeds using `matplotlib.pyplot`, and use our in-house least squares method to plot a linear best-fit line (see Appendix D for the least squares code). We also again calculate an R^2 value for the best-fit line to verify the accuracy and closeness of our best-fit line using our in-house method, for which the code can be found in Appendix E.

The framework code for data processing in this section was also developed by myself, and shared with other group members for their use and modification.

4. DATA ANALYSIS & MODELING

4.1. Read Noise and Conversion Gain Results

The subtraction of the mean dark frame brightness from every pixel in the light frames ensures that the only source of noise is read noise (σ_R); any other noise (primarily dark current and shot noise) is present in both the light

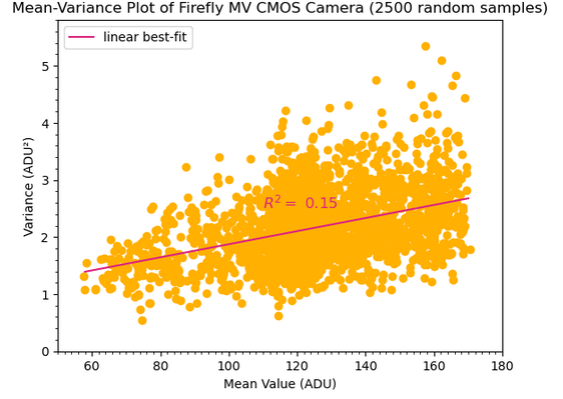


Figure 4. Mean-variance plot of 2,500 random samples from diff frames, with linear best-fit line.

frames and the average dark frame brightness and is therefore canceled out when subtracting one from the other.

Figure 4 presents the mean-variance plot with a scatter of values from 2500 randomly-selected pixels. A linear best-fit line is plotted above the data using our in-house least squares method. The slope of the best-fit line is 0.011 ADU, and its intercept 0.728 ADU². We also obtain an R^2 value of 0.15, calculated using an in-house method (see Appendix E).

The camera’s conversion gain (G , in e^-/ADU) is given by the reciprocal of the slope, and read noise (σ_R , in ADUs) by the square root of the y-intercept multiplied by G . Therefore, we calculate G :

$$G = 0.011^{-1} = 87.2 \text{ } e^-/\text{ADU} \quad (3)$$

And the read noise:

$$\sigma_R = \sqrt{0.738} \cdot G = 74.4 \text{ } e^- \quad (4)$$

It is important to note however that these results have some flaws, namely with the camera blur and very large spread of the data (including the very low R^2 value). These are addressed in Section 5.1.

4.2. Dark Current Results

Figure 5 presents the scatter plot of aggregate mean brightnesses (in ADU) of our CMOS camera at shutter speeds of 0.001s, 0.005s, 0.010s,

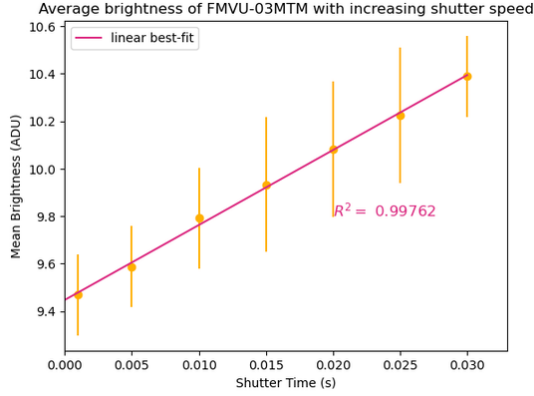


Figure 5. Scatter plot of the mean brightness of the FMVU-03MTM CMOS camera at increasing shutter speeds, with linear best-fit line.

0.015s, 0.020s, 0.025s, 0.030s, as well as the linear regression line plotted using our least squares method. We calculate $R^2 = 0.99762$.

The camera’s dark current from individual pixels is given by the slope, in ADU/s/px. To find the dark current in terms of photoelectrons, we multiply the slope by the camera’s gain. With a slope of 31.6 ADU/s/px, we find the camera’s dark current (σ_{DC}) using the result of G from Equation 3:

$$\sigma_{DC} = G \cdot 31.6 = 2,753 \text{ e}^- / (s \cdot px) \quad (5)$$

The best-fit is extremely strong ($R^2 > 0.99$) but these results raise questions regarding the uncertainty of the data points. These topics are discussed in Section 5.2.

5. DISCUSSION

5.1. Read Noise and Gain

Our results for G and σ_R from Section 4.1 are $87.2 \text{ e}^-/\text{ADU}$ and 74.4 e^- (or 0.859 ADUs). We can estimate a full-well capacity of 22.2 ke^- (multiplying G by 255, our maximum ADU value), which is nearly on par with the KAF-8300 commercial CCD typical values presented in slide 51 of the September 15 lecture ($>25.5 \text{ ke}^-$). Our σ_R is however larger by a factor greater than 4.5 compared to the value for the commercial CCD from the same slide (16 e^-).

This pair of observations seems rather surprising, as CMOS cameras are typically more performant than CCD sensors. This could however also be linked to intrinsic differences in cost and material/hardware quality, for which little information could be found online.

This part of the lab is also not without flaws; two main issues arise from the method and results. First, blurring the camera’s image to obtain a broader range of ADU values may have affected the original distribution of brightnesses; indeed, the blur causes most pixels to agglomerate to an average grey value in the 120 ADU range, which explains the significantly larger number of points in that area of Figure 4. Less blur could lead to a broader range of data and, thus, a better fit and clearer relationship.

Second, we notice that the scatter plot has a general increasing trend, but that points at all ADU values have widely spread variance values. This is confirmed by the extremely low R^2 value of 0.15 we obtain. There are two likely causes of this occurrence: first, the prevalence of this pattern in the 120 ADU range may suggest that camera blur was at play, meshing brightnesses together. Second, the room’s lights combined with our selected framerate could play a role in such a large spread of variances: the lights in the SS1072 room most likely flicker at 120Hz (as this experiment was performed in Canada where the power line frequency is 60Hz), compared to our framerate of 45.6fps. This likely affects photoelectron detections significantly between frames, as the camera detects different timings of the lights’ flicker much more than if the framerate had been set to a factor of 120.

Future work, if performed indoors, would ensure that the light sources in the room flicker at a rate that is a multiple of the chosen framerate, to limit these effects. The camera’s target would also be more appropriately selected with a broader and more consistent range of ADU values without relying on blur.

5.2. Dark Current

The results from Section 4.2 are certainly noteworthy, but are not without their drawbacks. We find a dark current of approximately $2,753 \text{ e}^- \text{s}^{-1} \text{px}^{-1}$ from our linear best-fit on the mean brightness over shutter time plot (Figure 5). These results are particularly low compared to the CCD dark current over temperature curve given on slide 47 of the September 15 lecture: the plot estimates a dark current in the 10^7 order of magnitude, while ours is 3 orders lower. This could be linked to the fact that CMOS cameras are usually much less affected by dark current than CCD sensors, but also due to the large uncertainties in our mean ADU measurements.

The large amplitude of the error bars is linked to the number of samples taken for each exposure time: the maximum number of used samples for one shutter speed is 5, so the largest reduction of σ_R is a division by $\sqrt{5}$ by Equation 2. This factor is even lower for 10ms, 15ms, 20ms, and 25ms exposure time samples because of discarded outliers: the factor is 2 for the 10ms exposure time, and $\sqrt{3}$ for all others.

We note that the best-fit's R^2 value is exceptionally strong (>0.99), inferring a near-certainty of the linear relationship between mean brightness of dark frames and shutter time. The slight divergence of some points from the best-fit line cannot have been caused by an improperly-placed or imperfect lens cap, as it was not moved at any point during the experiment; this would be a source of systematic uncertainty rather than the random one we observe on the scatter plot. The most likely explanation is the camera's read noise because of the variations' randomness.

Else, it is possible that the temperature of the camera shifted slightly over the process of data collection. Indeed, dark current is proportional to temperature as it is simply thermal excitation of electrons causing additional reads from the

camera. This means that the proportionality follows the same relationship as the Boltzmann distribution:

$$\sigma_{DC} \propto e^{-\frac{1}{T}} \quad (6)$$

Where T is the camera's temperature in K. Our T is really high ($\approx 298\text{K}$, room temperature) meaning that small differences in temperature are unlikely to have major effects. However, our values shift from the best-fit line by very small values, so camera temperature cannot be ruled out as a cause of random error.

Finally, the most puzzling part of the original data is the existence of frames that are almost completely dark (see the outliers in Table 1): these frames have extremely low mean brightness of the 10^{-2} order of magnitude. Such a mean implies that all pixels effectively “shut down” during the data recording, and pick up little to no photoelectrons. Since this is a CMOS camera, which converts charge to voltage at every pixel rather than at one global output node (which is how CCDs operate), it is also extremely unlikely that the brightness values were lost in conversion, as all pixels would have to have this same error simultaneously in one frame. The number of outlier frames only makes this idea more unlikely.

One guess is that the source of these outliers was the capture program itself. However, this does not appear likely because of the fact that there were no such issues with the light and dark frame captures when finding σ_R . It can be pointed out that different cameras were used between those measurements and the ones from this section, so it is possible that the camera-software combination might have been at play, with improper communication between the two parties causing some frames to not have their full exposure time, for instance. Future work might consider investigating the camera internals and/or the capture program's source code, to identify where these anomalies could have originated from.

APPENDIX

A. HANDOUT FIGURE RECREATION CODE

The code below demonstrates the `matplotlib.pyplot` operations performed to recreate the graph. `p1_spl` is the hardcoded array of data values provided in the handout, and its associated mean is `p1_mean`. The `factorial` call in line 17 is to the method implemented in `scipy.special`.

```
ax = plt.axes()
plt.hist(p1_spl, bins=17, range=(5,22), edgecolor="black", color="white")
plt.xlim(5, 23)

ax.set_yticks(np.arange(0, 5, 1))
ax.set_xticks(np.arange(5, 25, 5))

ax.set_yticks(np.arange(0, 4.6, 0.1), minor=True)
ax.set_xticks(np.arange(5, 23, 1), minor=True)

plt.xlabel('Photon Rates')
plt.ylabel('Number of Measurements')
plt.title('Photon count rates (recreation of Figure 7 from the lab handout)')

# Poisson distribution overplot
x = np.arange(6.5, 21.5, 0.1)
y = np.exp(-p1_mean) * np.power(p1_mean, x)/factorial(x)

amp = 31 # amplify the presence of the Poisson distribution
ax.plot(x, y * amp, color="gray", linestyle="dashed")
```

B. POISSON PMF AND GAUSSIAN PDF IN-HOUSE METHODS

The following code presents the in-house methods implemented by myself to calculate the Poisson pmf and Gaussian pdf, using NumPy. The Poisson distribution implements a loop division method for the factorial denominator to avoid integer overflows with the second plot's values (named `p2_large`).

```
def pois(x_arr, mu):
    """
    Return Poisson distribution using integer array x_arr and mean mu.
    Used to counteract scipy.special.factorial() overflow for p2_large.
    """
    result = np.exp(-mu, dtype=np.float128) \
        * np.power(mu, x_arr, dtype=np.float128)

    # division by factorial, implemented by integer
```

```

    for i in range(0, len(result) - 1):
        for j in range(1, x_arr[i] + 1):
            result[i] /= j
    return result

def gauss(x_arr, mu, sd):
    """
    Return Gaussian distribution of x_arr with mean mu and standard
    deviation sd.
    """
    return np.exp(-0.5 * ((x_arr - mu)/sd)**2)/(sd * (2*np.pi)**0.5)

```

C. SETTINGS.TXT FILES FROM DARK (LEFT) AND LIGHT (RIGHT) FRAME SETS

Below are the full `settings.txt` files generated with the sets of 30 dark (left) and light (right) images captured for the read noise and gain measurements (Section 2.5). The files include camera parameters, mean ADU, standard deviation, and minimum and maximum values over all recorded frames. These have been slightly modified from their original format to be readable in two columns.

AST325/326 Lab 1 acquisition	AST325/326 Lab 1 acquisition
timestamp : 2026-09-24T18:49:17.387390 -04:00	timestamp : 2026-09-24T18:51:04.107489 -04:00
camera : Point Grey Research Firefly MV FMVU-03MTM	camera : Point Grey Research Firefly MV FMVU-03MTM
GUID : 0x00B09D0100B9DB18	GUID : 0x00B09D0100B9DB18
mode : 752x480 MON08 FORMAT7	mode : 752x480 MON08 FORMAT7
frames : 30	frames : 30
auto controls: OFF	auto controls: OFF
sensor : monochrome	sensor : monochrome
pixels saved: all	pixels saved: all
image size : 752 x 480 pixels	image size : 752 x 480 pixels
settings as read back from the camera:	settings as read back from the camera:
brightness = 180 (raw)	brightness = 180 (raw)
exposure = 40 (raw)	exposure = 40 (raw)
frame_rate = 45.6279 fps	frame_rate = 45.6279 fps
gain = 0 dB	gain = 0 dB
gamma = 0 (raw)	gamma = 0 (raw)
shutter = 0.00541598 s	shutter = 0.00541598 s
trigger = 0 (raw)	trigger = 0 (raw)
trigger_delay = 0	trigger_delay = 0
frame means: mean=131.928 std=0.433 min=131.368 max=132.643	frame means: mean=7.856 std=0.004 min=7.849 max=7.865

D. LEAST-SQUARES REGRESSION IN-HOUSE METHOD

The following code is our in-house least-squares regression function, implemented by myself. It takes as arguments the data arrays `x_arr`, `y_arr`, and the number of samples `N`, and returns the slope `m` and intercept `c` of the linear best-fit. This is done by substituting array values into the analytical solution to the optimization problem of least-squares regression, which is an inverse-matrix and vector multiplication:

$$\begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} \sum x_i^2 & \sum x_i \\ \sum x_i & N \end{bmatrix}^{-1} \begin{bmatrix} \sum x_i y_i \\ \sum y_i \end{bmatrix} \quad (\text{D1})$$

Where x_i, y_i are terms from the first and second array respectively. We refer to the determinant as `det` in our code.

```
def least_squares(x_arr, y_arr, N):
    """
    Return a tuple of gradient (m) and constant (c) for linear regression
    using values from arrays x_arr and y_arr.
    Both arrays contain N entries.
    """
    x_arr2 = x_arr ** 2
    xy_arr = x_arr * y_arr

    sum_x = np.sum(x_arr)
    sum_y = np.sum(y_arr)
    sum_x2 = np.sum(x_arr2)
    sum_xy = np.sum(xy_arr)

    det = (N * sum_x2 - sum_x ** 2)**(-1)

    m = det * (sum_xy * N - sum_y * sum_x)
    c = det * (sum_y * sum_x2 - sum_xy * sum_x)

    return (m, c)
```

E. R^2 IN-HOUSE METHOD

The following code is our in-house coefficient of determination R^2 function, implemented by myself with review and discussion with the rest of the group. This function takes in the two data arrays (`x_arr`, `y_arr`) and returns the coefficient of determination of the best-fit line using the covariance of the two arrays, calculated with the covariance function from NumPy. This method is closely linked to the R value calculation in `scipy.stats.linregress`, as our code is a simplified version of it which works in the context of this lab.

```
def r_squared(x_arr, y_arr):
    """
    Return the R-squared value of a best-fit line of y_arr against x_arr,
    using covariance results from numpy.cov.
    """
    sumx_2, sumxy_2, sumy_2 = (np.cov(x_arr, y_arr, bias=True)[0][0],
                               np.cov(x_arr, y_arr, bias=True)[0][1],
                               np.cov(x_arr, y_arr, bias=True)[1][1])

    if sumx_2 * sumy_2 == 0: # zero denominator of R^2
        return 0
    else:
        return (sumxy_2 ** 2) / (sumx_2 * sumy_2)
```

F. EXCERPT FROM SETTINGS.TXT FILES OF FRAME SETS AT VARYING SHUTTER SPEEDS

The following excerpt is from the `settings.txt` file generated alongside the set of 5 captures made at 1ms shutter speed in Section 2.6. The only parameter that changes between files is shutter speed, increasing through 1ms, 5ms, 10s, 15ms, 20ms, 25ms, and 30ms. Note that there is a very minor rounding error on the shutter speed which is intrinsic to the viewer program, but is not considered significant for our measurements because of its size. All settings other than the shutter speed were verified to be the same between capture sets. The mean values at the bottom of the file are not presented as they vary between capture sets, and particularly because of the outliers in the values for 10ms, 15ms, 20ms, and 25ms shutter speeds which create large anomalies in these values. The full files for each capture set are available in the main branch of our GitHub repository,³ under the P1/src/4-2-5 folder in their respective shutter speed folders.

```
AST325/326 Lab 1 acquisition
timestamp   : 2026-10-01T18:23:30.478704-04:00
camera      : Point Grey Research Firefly MV FMVU-03MTM
GUID        : 0x00B09D0100B9DB20
mode        : 752x480 MONO8 FORMAT7
frames      : 5
auto controls: OFF
sensor      : monochrome
pixels saved: all
image size  : 752 x 480 pixels

settings as read back from the camera:
  brightness = 146 (raw)
  exposure   = 7 (raw)
```

³ <https://github.com/flypt7/AST326-Practicals>

```
frame_rate    = 30.0004 fps  
gain          = 12.0412 dB  
gamma         = 0 (raw)  
shutter       = 0.000999868 s  
trigger       = 0 (raw)  
trigger_delay = 0
```